



T-201-GSKI Data Structures

In-Class Hands-On: Reflections on Data Structures – with Solutions

Work through each exercise as a group (2-3 students), discussing your reasoning before writing up your answers. *Please close all laptops and put phones away!* If you need help, raise your hand and a teaching assistant (TA) will come to you.

The goal is genuine thinking and discussion — not necessarily getting every answer right. Any reasonably thoughtful submission receives full credit.

Write your final answers clearly on a single sheet of paper (both sides are fine; ask a TA for a second sheet if needed). Write the full name of every group member at the top. Hand your sheet(s) to a TA when done.

Instructions

Several of your fellow students in the GSKI class are making the statements below. For each one, engage with it using the following two-step approach:

1. **Acknowledge** what is correct or reasonable in the statement
2. **Refute** — if you disagree — by explaining what the statement gets wrong or oversimplifies.

This approach is called a **steelman argument**: rather than dismissing a claim outright, you first reconstruct it in a stronger form before pushing back. This forces more careful thinking and guards against shallow rebuttals.

Example

Statement:

"A hash table gives $O(1)$ average lookup, so there's no reason to ever use a BST for implementing a map."

Response:

"It is true that for heavy workloads with no ordering requirements, a well-implemented hash table will outperform a BST in practice. However, a BST-based map also has advantages: it preserves key order and thus supports operations like range queries and in-order traversal, which a hash table cannot efficiently provide."

Statements to respond to:

1. *"If two data structures have the same worst-case complexity, it doesn't matter which one to pick."*
2. *" $O(1)$ amortized update for dynamic means every single append is $O(1)$. Right?"*
3. *"Linked lists are pointless — a dynamic array does everything they do and gives you random access too. Why would anyone ever use a linked list?"*
4. *"Doubly linked lists just waste memory storing an extra pointer. A singly linked list is always the better choice."*
5. *"A queue is strictly preferable to a stack. Given a free choice between the two, you should always pick the queue."*
6. *"The 'map' ADT is just a fancy name for a hash table. They're the same concept."*
7. *"Two hash tables with different hash functions but the same collision resolution strategy will perform identically."*
8. *"The load factor of a hash table doesn't matter much as long as your hash function is good."*

1. *"If two data structures have the same worst-case complexity, it doesn't matter which one to pick."*

It's true that worst-case complexity is important. If two structures both guarantee, say, $O(n)$ worst-case time for an operation, they may appear equivalent. However, (worst-case) time complexity does not tell the entire story; for example, one might be a constant factor (10, 100, 1000, or worse) times slower than the other. Also, for many problems average-time complexity might be more important than worst case.

2. *" $O(1)$ amortized update for dynamic means every single append is $O(1)$. Right?"*

It's understandable to interpret amortized $O(1)$ this way, since most append operations take a constant time. However, amortized $O(1)$ means that *on average over many operations* the append operation costs constant time. Occasional individual operations can, however, still be expensive — for example, when the array resizes and copies all elements, that single append takes $O(n)$. So not every append is $O(1)$.

3. *"Linked lists are pointless — a dynamic array does everything they do and gives you random access too. Why would anyone ever use a linked list?"*

Dynamic arrays are very versatile. They provide fast indexing ($O(1)$) and are often more memory-efficient and cache-friendly, which makes them a strong default choice in many situations. However, linked lists still have some advantages. They allow $O(1)$ insertion and deletion, without shifting elements as arrays do. Thus, when the usage calls for a frequent insertions/removals in the middle of a sequence, they are generally the preferred choice.

4. *"Doubly linked lists just waste memory storing an extra pointer. A singly linked list is always the better choice."*

It's true that doubly linked lists use more memory per node due to the extra (prev) reference, but this overhead only matters in very memory-constrained settings. Overall, doubly linked lists allow more flexible traversal (e.g., reverse) and $O(1)$ deletion of a node anywhere in the list. This added flexibility outweighs the memory cost, so doubly linked list are often preferable.

5. *"A queue is strictly preferable to a stack. Given a free choice between the two, you should always pick the queue."*

Queues are preferred for many real-world scenarios. However, stacks and queues serve fundamentally different purposes. Choosing one over the other depends on the required access pattern (LIFO vs FIFO), not preference. Neither is strictly better.

6. *"The 'map' ADT is just a fancy name for a hash table. They're the same concept."*

Hash tables are a common implementation of maps, and in some programming languages, "map" often refers to a hash-based structure. However, a map is an *abstract data type (ADT)* that defines key-value storage and lookup *behavior*, while a hash table is just one implementation. Maps can also be implemented using, for example, balanced trees, which provide ordering and $O(\log n)$ operations. So they are not the same concept.

7. *"Two hash tables with different hash functions but the same collision resolution strategy will perform identically."*

Collision resolution strategy (like separate chaining) plays an important role in determining performance, so it's reasonable to think it dominates behavior. However, the hash function is equally or even more critical. A poor hash function can cause many collisions, degrading performance significantly, even with a good resolution strategy. A well-designed hash function distributes keys evenly, which is essential for achieving expected $O(1)$ performance. So different hash functions can lead to very different outcomes.

8. *"The load factor of a hash table doesn't matter much as long as your hash function is good."*

A good hash function does help distribute keys evenly, which is very important for performance. However, that alone is not sufficient and load factor still matters a lot. As the table fills up, collisions become more frequent regardless of hash quality, increasing lookup time. That's why hash tables resize when the load factor crosses a threshold. So it's a key parameter, not an insignificant detail.